

به نام خدا

طراحی الگوریتم‌ها

آرش شفیعی



جستجوی ترکیبیاتی

- روش‌های برنامه‌ریزی پویا و حریصانه را برای حل دسته‌ای از مسئله‌های بهینه‌سازی بررسی کردیم. در مسئله‌های بررسی شده، یک راه حل ساده برای پیدا کردن جواب مسئله جستجوی همه حالت‌ها است ولی چنین جستجویی در زمان معقول (زمان چند جمله‌ای) امکان پذیر نیست چرا که تعداد همه حالت‌های ممکن از مرتبه نمایی است و بنابراین جستجوی همه حالت‌ها در زمان نمایی صورت می‌گیرد. توسط روش‌های برنامه‌ریزی پویا و حریصانه روش‌هایی برای حل مسئله‌ها در زمان معقول ارائه کردیم.
- پیدا کردن جواب در زمان چند جمله‌ای برای همه مسئله‌ها همیشه امکان پذیر نیست و گاهی تنها راه حل، جستجوی فضای حالت برای آن مسئله است. جستجوی فضای حالت به معنی تولید همه حالت‌های جواب مسئله و انتخاب حالت بهینه یا حالت مورد نظر از بین همه حالت‌های ممکن است. تعداد این حالت‌ها از مرتبه نمایی است و بنابراین چنین الگوریتم‌هایی به زمان نمایی برای محاسبه نیاز دارند و در نتیجه برای مسائلی با اندازه بزرگ در عمل قابل استفاده نیستند.

جستجوی ترکیبیاتی

- در مبحث جستجوی ترکیبیاتی¹، روش‌های جستجوی همه فضای حالت برای یک مسئله بررسی می‌شود. در این روش‌ها مطالعه می‌کنیم چگونه به طور منظم همه حالت‌ها را بررسی کنیم و یا اینکه چگونه با استفاده از روش‌هایی فضای حالت را محدود کنیم.
- ترکیبیات² شاخه‌ای از ریاضیات است که در آن به بررسی ساختارهای متناهی و شمارش این ساختارها می‌پردازیم.
- برای مثال گراف یک ساختار متناهی است و تعداد مسیرها در یک گراف، متناهی است. برای پیدا کردن بلندترین مسیر در یک گراف می‌توانیم همه مسیرها را بررسی و بلندترین آنها را انتخاب کنیم.
- بهینه‌سازی ترکیبیاتی³ شاخه‌ای از بهینه‌سازی است که در آن مجموعه حالت‌ها گسسته است و هدف پیدا کردن جواب بهینه از بین همه حالت‌های ممکن است.

¹ combinatorial search

² combinatorics

³ combinatorial optimization

- روش پسگرد¹ روشی است که با استفاده از آن همه فضای حالت را جستجو می‌کنیم. در این روش با یکی از حالت‌ها در فضای حالت شروع می‌کنیم و برای انتخاب حالت بعد یکی از پارامترهای فضای حالت را تغییر می‌دهیم. ممکن است در انتخاب حالت بعد، چند پارامتر قابل تغییر باشند. در اینصورت یکی از پارامترها را تغییر می‌دهیم و به حالت بعد می‌رویم و سپس پسگرد می‌کنیم تا یکی دیگر از پارامترها را تغییر دهیم. بدین صورت همه حالت‌ها در فضای حالت بررسی می‌شوند.

¹ backtrack

- فرض کنید وارد یک باغ پر پیچ و خم یا باغ هزارتو¹ شده‌اید و می‌خواهید راه خروجی را پیدا کنید. راه را در پیش می‌گیرید و به هر چند راهی که می‌رسید راه اول از سمت چپ را انتخاب می‌کنید. در پایان یا راه خروجی را پیدا می‌کنید و یا به بن‌بست بر می‌خورید. در صورتی که به بن‌بست رسیدید، باز می‌گردید تا به اولین چندراهی قبل از بن‌بست برسید. به جای راه اول از سمت چپ، دومین راه از سمت چپ را امتحان می‌کنید و در صورت برخورد با بن‌بست راه را باز می‌گردید و در چند راهی راه سوم را امتحان می‌کنید. فرض کنید که همهٔ راه‌ها در چند راهی آخر را امتحان کردید و به بن‌بست خوردید. در این صورت باید مسیر را بازگردید تا به دومین چند راهی قبل از بن‌بست برسید و این بار در دومین چندراهی قبل از بن‌بست، دومین راه را انتخاب کنید. این روند را ادامه می‌دهید تا راه خروجی را پیدا کنید. به این روش حل مسئله روش پسگرد گفته می‌شود.
- هر یک از چندراهی‌ها یکی از پارامترهای مسئله است که مقادیر مختلف آن را امتحان می‌کنید.

¹ maze

- روش پسگرد وقتی استفاده می‌شود که می‌خواهیم مسئله‌ای را حل کنیم که در آن عناصر یک دنباله¹ باید از اشیائی از یک مجموعه² معین انتخاب شوند، به طوری که دنباله ویژگی مشخصی داشته باشد و معیار³ معینی را برآورده کند.

¹ sequence

² set

³ criterion

مسئله چند وزیر

- می‌خواهیم تعداد n وزیر را در یک صفحه شطرنج $n \times n$ به گونه‌ای قرار دهیم که هیچ یک از وزیرها همدیگر را تهدید نکنند. به عبارت دیگر هیچ دو وزیری نباید در یک سطر یا ستون یا خط مورب مشترکی قرار بگیرند.
- دنباله‌ای که در این مسئله به دنبال آن می‌گردیم، دنباله‌ای است از n مکان که n وزیر در آنها قرار گرفته‌اند. مجموعه‌ای که هر یک از عناصر دنباله می‌توانند از اعضای آن مجموعه انتخاب شوند عبارت است از مجموعه‌ای شامل n^2 مکان بر روی صفحه شطرنج.
- ویژگی معینی که این دنباله باید داشته باشد این است که هیچ دو مکان انتخاب شده‌ای بر روی یک خط افقی، عمودی یا مورب قرار نگیرد.
- مسئله چند وزیر یا n وزیر، حالت کلی مسئله ۸ وزیر است که در آن ۸ وزیر در یک صفحه شطرنج استاندارد با تعداد 8×8 مکان قرار می‌گیرند.
- در اینجا برای سهولت نمایش حالات مختلف مسئله ۴ وزیر را در نظر می‌گیریم.

- روش پسگرد در واقع روشی است که در یک درخت¹ به صورت عمق-اول² جستجو می‌کند. پس ابتدا جستجوی عمق-اول در درخت را توضیح می‌دهیم.
- فرض کنید درختی داریم که از تعدادی رأس و یال تشکیل شده است و می‌خواهیم همه مسیرهای درخت را که با ریشه شروع می‌شوند و با یک برگ پایان می‌یابند بررسی کنیم.
- در واقع دنباله‌هایی که در اینجا بررسی می‌کنیم دنباله‌هایی هستند که یک مسیر در درخت را نشان می‌دهند و با ریشه آغاز و با یکی از برگ‌ها پایان می‌یابند.

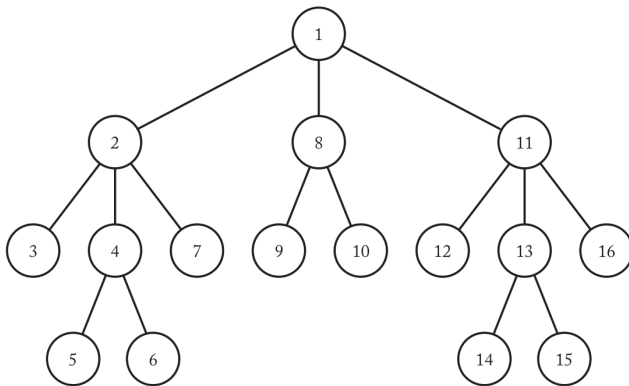
¹ tree

² depth-first

- ابتدا به سراغ ریشه درخت می‌رویم و ریشه را به عنوان اولین عنصر دنباله انتخاب می‌کنیم. سپس اولین فرزند از سمت چپ در سطح یک درخت را انتخاب می‌کنیم و اگر این رأس دارای فرزند بود اولین فرزند آن را در سطح دو انتخاب می‌کنیم. این کار را ادامه می‌دهیم تا به یک برگ برسیم. تا اینجا یک مسیر در درخت را بررسی کرده‌ایم. با فرض اینکه برگ در سطح n قرار دارد، به رأس پدر در سطح $n - 1$ باز می‌گردیم و فرزند دوم را انتخاب می‌کنیم. این کار را ادامه می‌دهیم تا مسیر دوم و به همین ترتیب همه مسیرها در درخت را بررسی کنیم.

مسئله چند وزیر

- درخت زیر را در نظر بگیرید. با استفاده از روش پسگرد ابتدا مسیر $(1, 2, 3)$ سپس $(1, 2, 4, 5)$ سپس $(1, 2, 4, 6)$ ، و به همین ترتیب $(1, 2, 7)$ ، $(1, 8, 9)$ ، $(1, 8, 10)$ ، الی آخر بررسی می‌شوند.



- الگوریتم زیر این روش پسگرد را نشان می دهد.

Algorithm Depth First TreeSearch

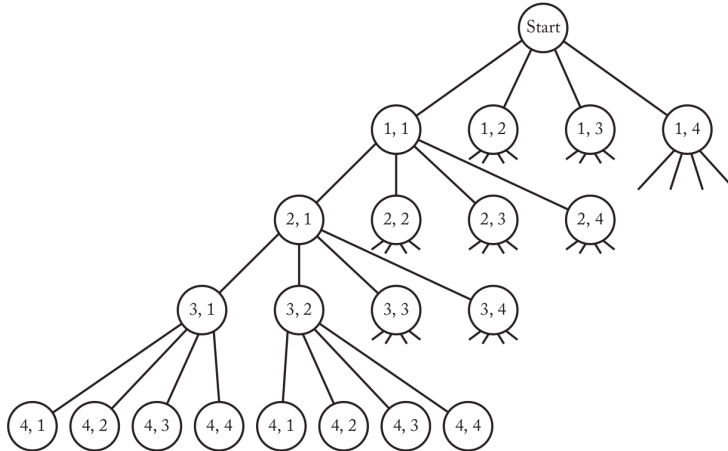
```
function DEPTH-FIRST-TREE-SEARCH(node v)
1: visit(v)
2: for each child u of v do
3:   Depth-First-Tree-Search(u)
```

مسئله چند وزیر

- حال که با روش پسگرد آشنا شدیم، مسئله ۴ وزیر را در نظر می‌گیریم. می‌خواهیم ۴ وزیر را در یک صفحه شطرنج 4×4 به گونه‌ای قرار دهیم که هیچ دو وزیری یکدیگر را تهدید نکنند. از آنجایی که هیچ دو وزیری نمی‌توانند در یک سطر قرار بگیرند، پس هر وزیر را باید در یک سطر متفاوت از بقیه وزیرها قرار دهیم. از آنجایی که هر وزیر در هر یک از ستون‌ها می‌تواند قرار بگیرد، بنابراین تعداد همه حالت‌هایی که باید بررسی شوند برابر است با $4 \times 4 \times 4 \times 4 = 256$.
- برای بررسی همه حالت‌ها درختی تشکیل می‌دهیم که در آن، مکان اولین وزیر در سطح یک انتخاب می‌شود و مکان وزیر دوم در سطح دوم و به همین ترتیب مکان وزیر سوم در سطح سوم و مکان وزیر چهارم در سطح چهارم انتخاب می‌شوند.
- یک مسیر از ریشه تا برگ درواقع یکی از گزینه‌های انتخاب مکان‌های صفحه را نشان می‌دهد. یک گزینه جواب مسئله است اگر در آن هیچ دو وزیری یکدیگر را تهدید نکنند.
- به درخت تشکیل شده درخت فضای حالت گفته می‌شود.

مسئله چند وزیر

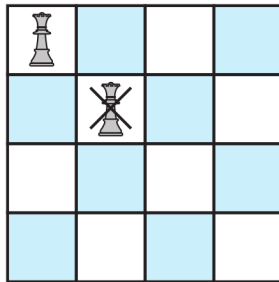
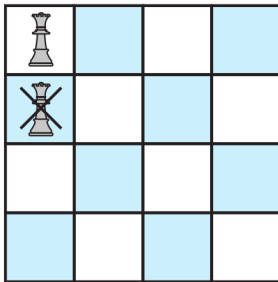
- قسمتی از درخت فضای حالات مسئله ۴ وزیر در شکل زیر نمایش داده شده است.



- این درخت در مجموع ۲۵۶ برگ دارد که هر مسیر از ریشه تا یکی از برگ‌ها یکی از گزینه‌ها را نشان می‌دهد. در هریک از رأس‌های درخت یک جفت (i, j) ذخیره می‌شود که برابر با یک مکان در صفحه شطرنج در سطر i و ستون j است.

مسئله چند وزیر

- جستجو در این درخت می تواند بهینه تر از بررسی همه ۲۵۶ انتخاب نیز انجام شود. برای مثال وقتی وزیر اول در مکان (1, 1) قرار گرفت، بدیهی است که وزیر دوم نمی تواند در مکان (2, 1) قرار بگیرد پس این مسیر در درخت ادامه داده نمی شود. همینطور وزیر دوم نمی تواند در مکان دوم قرار بگیرد پس نیازی نداریم این مسیر را نیز ادامه دهیم.
- در شکل زیر این بهینه سازی نشان داده شده است.



- پسگرد عبارت است از روندی که توسط آن وقتی در یک انتخاب به یک گزینه غیر جواب می‌رسیم، در درخت فضای حالت به عقب بازمی‌گردیم یا به عبارت دیگر از انتخاب یک رأس صرف نظر می‌کنیم و به رأس پدر پسگرد می‌کنیم تا فرزند بعدی پدر را انتخاب کنیم.
- به یک رأس در درخت فضای حالت نومید کننده¹ می‌گوییم، اگر انتخاب آن رأس به جواب نیانجامد و به همین ترتیب به یک رأس امید دهنده² می‌گوییم اگر با انتخاب آن همچنان احتمال رسیدن به جواب وجود داشته باشد.

¹ nonpromising

² promising

- به طور خلاصه در روش پسگرد، بر روی درخت فضای حالت، جستجوی عمق اول انجام می‌دهیم و در فرایند جستجو اگر به رأس نومید کننده برخورد کردیم مسیر را ادامه نمی‌دهیم و به رأس پدر پسگرد می‌کنیم.
- به این روش هرس کردن¹ فضای حالت نیز گفته می‌شود که در آن تعدادی از دنباله‌ها بررسی نمی‌شوند.

¹ pruning

- در حالت کلی این الگوریتم به صورت زیر نوشته می شود.

Algorithm Checknode

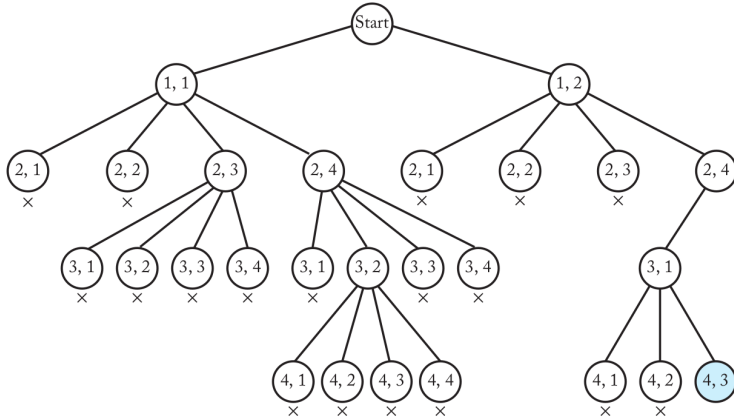
```
function CHECKNODE(node v)
1: if promising(v) then
2:   if there is a solution at v then
3:     write the solution
4:   else
5:     for each child u of v do
6:       Checknode(u)
```

- ریشه فضای حالت به تابع Checknode داده می شود که توسط آن رأس ریشه بررسی می شود. در بررسی یک رأس، ابتدا باید بررسی شود که انتخاب آن رأس امید دهنده است یا نومیید کننده. اگر انتخاب آن امید دهنده بود و به جواب رسیدیم، جواب را چاپ می کنیم. اگر انتخاب امید دهنده بود ولی به جواب نرسیدیم، رئوس فرزند به ترتیب برای بررسی انتخاب می شود.

- تابع promising برای مسئله‌های مختلف متفاوت است. در مسئله ۴ وزیر، این تابع مقدار نادرست باز می‌گرداند اگر مکان‌های انتخاب شده از ریشه تا رأس مورد بررسی، به صورت سطری، ستونی یا قطری در یک راستا باشند.

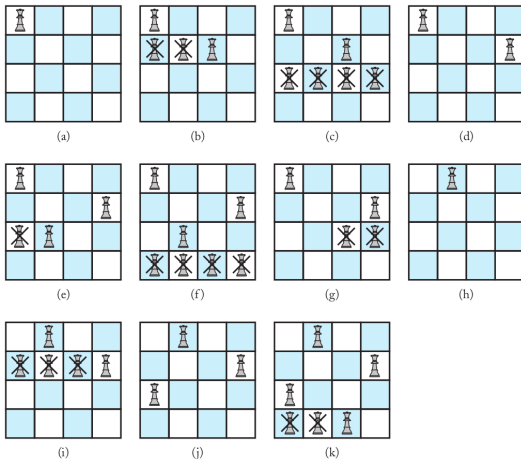
مسئله چند وزیر

- در شکل زیر قسمتی از درخت فضای حالت به صورت هرس شده نشان داده شده است.



مسئله چند وزیر

- در شکل زیر روند بررسی فضای حالت در صفحه شطرنج نشان داده شده است.



- تابع promising باید بررسی کند که آیا دو وزیر در یک ستون یا قطر قرار گرفته‌اند یا خیر.
- اگر $col(i)$ ستونی باشد که وزیر i ام در آن قرار گرفته است، باید بررسی کنیم که $col(i)$ و $col(j)$ برای هیچ دو وزیر i و j برابر نباشد.
- همچنین دو وزیر به صورت مورب یکدیگر را تهدید می‌کنند اگر $col(i) - col(j) = i - j$ و یا $col(i) - col(j) = j - i$ باشد.

- الگوریتم پسگرد برای مسئله چند وزیر به صورت زیر است. برای شروع الگوریتم تابع $Queens(0)$ فراخوانی می‌شود.

Algorithm Queens

```
function QUEENS(index i)
1: if Promising(i) then
2:   if i == n then
3:     print col[1] through col[n]
4:   else
5:     ▷ See if queen in (i+1)-st row can be
6:     ▷ positioned in each of the n columns.
7:     for j = 1 to n do
8:       col[i + 1] = j
9:       Queens(i + 1)
```

Algorithm Promising

– **function** PROMISING(index i)

 ▷ Check if any queen k threatens queen in the i-th row.

1: **for** k = 1 **to** i - 1 **do**

2: **if** col[i] == col[k] **or** |col[i] - col[k]| == i - k **then**

3: **return** false

4: **return** true

مسئله چند وزیر

- حال می‌خواهیم الگوریتم پسگرد برای چند وزیر را تحلیل کنیم. برای این کار تعداد رئوسی که در درخت فضای حالت بررسی می‌شوند را محاسبه می‌کنیم. از آنجایی که به دست آوردن تعداد دقیق حالات وقتی حالت‌ها هرس می‌شوند ساده نیست، برای تعداد رئوس بررسی شده در درخت فضای حالت یک کران بالا در نظر می‌گیریم.
- در درخت فضای حالات در سطح صفر، یک رأس داریم، در سطح یک تعداد n رأس، در سطح ۲ تعداد n^2 رأس و در سطح n تعداد n^n رأس داریم. پس تعداد همه رئوس بررسی شده برابر است با :

$$1 + n + n^2 + n^3 + \dots + n^n = \frac{n^{n+1} - 1}{n - 1}$$

- برای مسئله ۸ وزیر این تعداد برابر است با

$$\frac{8^{8+1} - 1}{8 - 1} = 19,173,961$$

مسئله چند وزیر

- حال یک کران بالای دیگر برای تعداد رئوس بررسی شده در نظر می‌گیریم. می‌دانیم که هیچ دو وزیری نمی‌توانند در یک ستون قرار بگیرند، بنابراین وقتی وزیر اول انتخاب شد، وزیر دوم تنها در ۷ مکان می‌تواند قرار بگیرد پس در سطح اول ۸ رأس و در سطح دوم ۸×۷ رأس، در سطح سوم $۸ \times ۷ \times ۶$ رأس داریم و بدین ترتیب الی آخر.

- پس در حالت کلی کران بالای رئوس بررسی شده برابر است با :

$$1 + n + n(n - 1) + n(n - 1)(n - 2) + \dots + n!$$

- برای مسئله ۸ وزیر این تعداد برابر است با $۱۰۹,۶۰۱$ رأس.

- تعداد دقیق رؤس بررسی شده را می‌توانیم با پیاده‌سازی الگوریتم به دست آوریم.
- از آنجایی که کران بالای تعداد حالت مورد بررسی $n!$ است، زمان اجرای الگوریتم پسگرد برای مسئله n وزیر $O(n!)$ است.

- روش شاخه و کران¹ همچون روش پسگرد درخت فضای حالت را برای یافتن جواب بررسی می‌کند.
- این روش معمولاً برای مسائل بهینه‌سازی استفاده می‌شود. در مسائل بهینه‌سازی هدف یافتن جواب بهینه (کوچکترین، بزرگترین، ...) است. در هر لحظه در هنگام پیمایش درخت فضای حالت یکی از جواب‌های به دست آمده تا آن لحظه بهینه است. بنابراین قبل از بسط دادن یک رأس می‌توانیم محاسبه کنیم آیا جوابی که با بسط دادن آن رأس به دست می‌آید، از جواب بهینه‌ای که تا آن لحظه به دست آمده است، بهتر است یا خیر. در صورتی که امیدی به یافتن جواب بهتر نبود، رأس مورد پیمایش بسط داده نمی‌شود.
- بنابراین اگر با بسط دادن یک رأس امید به یافتن جواب بهینه‌تر وجود نداشت، می‌گوییم آن رأس نومیدکننده² است، در غیر اینصورت امیددهنده³ است. یک الگوریتم شاخه و کران در هر رأس درخت جستجوی حالات، کرانی را محاسبه می‌کند که با استفاده از مقدار کران می‌توان گفت آیا آن رأس امید دهنده است یا خیر.

¹ branch and bound

² nonpromising

³ promising

- می‌توانیم مسئله کوله پشتی ۱-۰ را با استفاده از روش پسگرد حل کنیم.
- در سطح اول در درخت فضای حالت، دو حالت بررسی می‌شوند : (۱) کالای اول در کوله پشتی قرار می‌گیرد، یا (۲) کالای اول در کوله‌پشتی قرار نمی‌گیرد. همین‌طور در سطح دوم به ازای هر یک از دو حالت سطح اول درخت، باید دو حالت بررسی شوند : اینکه کالای دوم در کوله‌پشتی قرار می‌گیرد یا نمی‌گیرد. این فرایند ادامه پیدا می‌کند تا به یک جواب برسیم و هزینه کوله‌پشتی را محاسبه کنیم. در یک الگوریتم پسگرد باید همه برگ‌های درخت فضای حالت بررسی شده، برگ‌هایی که بیشترین هزینه را دارد انتخاب شود.

- همچنین در حل این مسئله می‌توان از روش شاخه و کران استفاده کرد. برای بررسی اینکه یک رأس امیددهنده است یا خیر، باید محاسبه کنیم که آیا حداکثر ارزشی که می‌تواند با برداشتن باقی اشیاء حاصل شود، از جواب بهینه به دست آمده تا آن لحظه بیشتر است یا خیر. تنها در صورتی بسط یک رأس را ادامه می‌دهیم که امیدی به بهبود جواب داشته باشیم.
- یک الگوریتم شاخه و کران به ازای هر رأس در درخت فضای حالت محاسبه می‌کند آیا هزینه‌ای که با بسط آن رأس به دست می‌آید از جواب بهینه به دست آمده بهتر خواهد بود یا خیر. به عبارت دیگر می‌گوییم در هر رأس یک کران محاسبه می‌شود و با استفاده از کران محاسبه شده تصمیم گرفته می‌شود رأس مورد بررسی هرس شود یا خیر.

- در روش شاخه و کران، با استفاده از مقدار کران به دست آمده، نه تنها می‌توان تصمیم گرفت که یک رأس بسط داده شود و یا خیر، بلکه می‌توان علاوه بر آن با استفاده از کران به دست آمده، تصمیم گرفت کدام رأس برای بسط دادن مناسب‌تر است.
- با استفاده از این روش معمولاً می‌توان با سرعت بیشتری به جواب بهینه دست پیدا کرد.
- به این روش جستجوی بهتر اول¹ با هرس کردن شاخه و کران² گفته می‌شود.

¹ best-first search

² branch and bound pruning

- در جستجوی بهتر اول، درخت فضای حالت را با استفاده از نوعی از جستجوی سطح اول¹ پیمایش می‌کنیم.
- در پیمایش سطح اول، ابتدا ریشه بررسی می‌شود، سپس رئوس سطح یک و پس از آن رئوس سطح دو و بدین ترتیب همه رئوس تا برگ‌ها بررسی می‌شوند.
- برخلاف جستجوی عمق اول که در آن از یک الگوریتم بازگشتی استفاده می‌شود، در جستجوی سطح-اول از یک صف برای پیمایش رئوس درخت استفاده می‌کنیم.
- بدین ترتیب فرزندان یک رأس در صف قرار می‌گیرند و به ترتیب فرزندان یک به یک از صف خارج شده و فرزندانشان پیمایش می‌شوند. بنابراین در این روش ابتدا سطح اول درخت، سپس سطح دوم و به همین ترتیب همه سطوح پیمایش می‌شوند. به همین دلیل به این جستجو سطح-اول گفته می‌شود.

¹ breadth-first search

- الگوریتم زیر الگوریتم جستجوی درخت با استفاده از روش سطح-اول را نشان می‌دهد.

Algorithm Breadth-First-Tree-Search

```
function BREADTH-FIRST-TREE-SEARCH(tree T)
1: queue Q
2: node u, v
3: initialize(Q)  ▷ initialize Q to be empty.
4: u = root(T)
5: visit(u)
6: enqueue(Q, u)
7: while !empty(Q) do
8:   v = dequeue(Q)
9:   for each child u of v do
10:     visit(u)
11:     enqueue(Q, u)
```

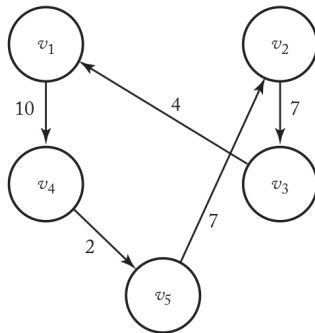
مسئله فروشنده دوره‌گرد

- یک فروشنده دوره‌گرد می‌خواهد برای فروش اجناس خود به همه شهرها سفر کند. هر دو شهر می‌توانند توسط یک جاده یک طرفه به یکدیگر متصل شده باشند و هر جاده طولی معین دارد. فروشنده دوره‌گرد می‌خواهد از شهر خود سفر را آغاز کند و مسیری را بپیماید که از هر شهر تنها یک بار عبور کند و در پایان به شهر خود بازگردد. در ضمن می‌خواهد مسیر پیموده شده کوتاهترین مسیر باشد.
- این مسئله را با استفاده از یک گراف مدلسازی می‌کنیم. در یک گراف جهت‌دار، می‌خواهیم کوتاهترین مسیری را پیدا کنیم که از یک رأس آغاز می‌شود، از هریک از رئوس تنها یک بار عبور می‌کند و به رأس اول باز می‌گردد. چنین مسیری یک مسیر بهینه است. از آنجایی که این مسیر بهینه از همه رئوس عبور می‌کند، بنابراین می‌توانیم از هر رأسی مسیر را آغاز کنیم. به مسیری که از هر رأس تنها یک بار عبور می‌کند مسیر همیلتونی می‌گوییم و به مسیری که از هر رأس تنها یک بار عبور کند و به رأس اول بازگردد یک دور همیلتونی می‌گویند. در اینجا به دنبال کوتاهترین دور همیلتونی می‌گردیم.

مسئله فروشنده دوره گرد

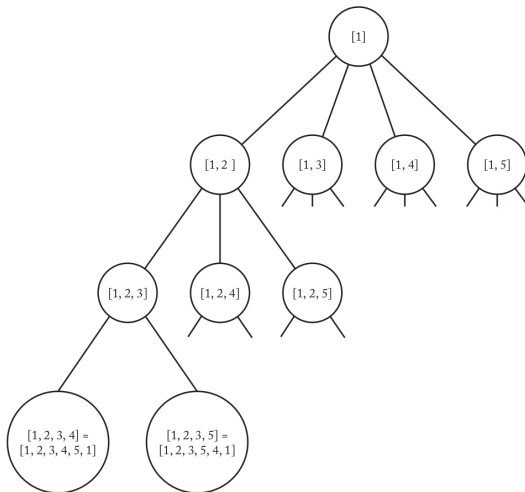
- در شکل زیر ماتریس مجاورت برای یک گراف جهت دار نشان داده شده است که در آن از هر رأس به رأس دیگر یک یال وجود دارد. اعداد در ماتریس مجاورت طول یال ها از یک رأس به رأس دیگرند. کوتاهترین دور همیلتونی در این گراف نشان داده شده است.

0	14	4	10	20
14	0	7	8	7
4	5	0	7	16
11	7	9	0	2
18	7	17	4	0



مسئله فروشنده دوره گرد

- قسمتی از درخت جستجوی فضای حالت برای این مسئله در زیر نشان داده شده است.



- در این درخت فضای حالت، با رأس شماره ۱ از گراف آغاز می کنیم. مسیر بهینه ممکن است از هر یک از رؤس ۲، ۳، ۴ و ۵ عبور کند، بنابراین به ازای هریک از این رؤس یک فرزند در سطح یک در درخت فضای حالت می سازیم. رأس [1, 2] در درخت فضای حالت مسیری را در گراف مشخص می کند که از رأس ۱ و ۲ در گراف عبور کند.

- اکنون باید برای هر رأس در درخت فضای حالت یک مقدار کران پیدا کنیم. در هر رأس درخت فضای حالت یک کران پایین برای طول مسیری که می توان با بسط دادن آن رأس به دست آورد محاسبه می کنیم.
- اگر کران پایین محاسبه شده در یک رأس از کوتاهترین مسیر همیلتونی به دست آمده تا آن لحظه کمتر باشد، آن رأس درخت فضای حالت امیددهنده است و آن رأس را بسط می دهیم، در غیر این صورت آن رأس نومیدکننده است و بسط دادن را از آن رأس ادامه نمی دهیم.

مسئله فروشنده دوره گرد

- برای محاسبه کران به صورت زیر عمل می کنیم.

- در هر رأس درخت فضای حالت تعدادی از رئوس گراف پیمایش شده و تعدادی پیمایش نشده اند. به ازای رأس های پیمایش شده در گراف طول مسیر معین شده است. اما به ازای رأس های پیمایش نشده در گراف باید یک کران پایین برای طول مسیر محاسبه کنیم.

- رأس پیمایش نشده V_i را در نظر بگیرید. جهت پیدا کردن یک کران پایین برای کوتاهترین مسیر، به ازای هریک از رأس های پیمایش نشده V_i باید کوتاهترین یال خروجی از آن رأس را به صورت $\min_k(V_i, V_k)$ محاسبه کنیم و طول کوتاهترین یال های خروجی را به ازای همه رئوس پیمایش نشده به صورت $\sum_i \min_k(V_i, V_k)$ محاسبه کنیم.

- به عبارت دیگر به ازای هر یک از رئوس پیمایش نشده V_i باید یالی را پیدا کنیم که از V_i خارج می شود و کمترین هزینه را دارد. مجموع طول این یال ها یک کران پایین برای طول مسیر است. توجه کنید که این بدین معنا نیست که کوتاهترین ها الزاما در مسیر همیلتونی انتخاب می شوند، بلکه بدین معناست که هیچ مسیر همیلتونی با طول کمتر از مقدار محاسبه شده وجود نخواهد داشت.

مسئله فروشنده دوره گرد

- محاسبه کران را با یک مثال بررسی می کنیم. ماتریس مجاورت زیر را در نظر بگیرید.

$$\begin{bmatrix} 0 & 14 & 4 & 10 & 20 \\ 14 & 0 & 7 & 8 & 7 \\ 4 & 5 & 0 & 7 & 16 \\ 11 & 7 & 9 & 0 & 2 \\ 18 & 7 & 17 & 4 & 0 \end{bmatrix}$$

- در ریشه درخت فضای حالت هیچ یک از رئوس گراف پیموده نشده اند. کران پایین مسیر در ریشه را می توانیم توسط رابطه $\sum_i \min_k(V_i, V_k)$ محاسبه کنیم. این مقدار برابر است با :

$$\sum_i \min_k(V_i, V_k) = 4 + 7 + 4 + 2 + 4 = 21$$

- بنابراین طول کوتاه ترین مسیر ممکن با شروع از رأس V_1 برابر است با ۲۱.

مسئله فروشنده دوره گرد

- حال به ازای همه فرزندان ریشه یک کران محاسبه می‌کنیم. برای مثال رأس $[1, 2]$ در درخت فضای حالت را در نظر بگیرید. یالی با طول 14 از رأس V_1 به رأس V_2 پیموده شده است. طول کوتاهترین مسیری که با ادامه این مسیر می‌تواند وجود داشته باشد برابر خواهد بود با ۱۴ به علاوه کوتاهترین مسیری که از بقیه رؤس به غیر از V_2 عبور می‌کند. توجه کنید که از آخرین رأس در مسیر که در اینجا V_2 است، نمی‌توانیم به اولین رأس در مسیر که در اینجا V_1 است، بازگردیم اما امکان این که از هر یک از رؤس دیگر به V_1 برویم وجود دارد.

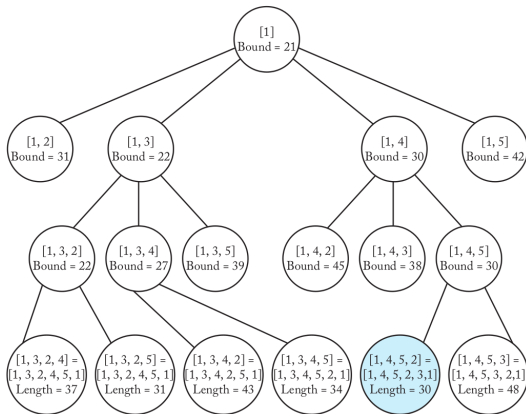
0	14	4	10	20
14	0	7	8	7
4	5	0	7	16
11	7	9	0	2
18	7	17	4	0

- کران پایین مسیر در این رأس برابر است با :

$$14 + \sum_{i \in \{2,3,4,5\}} \min_k(V_i, V_k) = 14 + 7 + 4 + 2 + 4 = 31$$

مسئله فروشنده دوره گرد

- بدین ترتیب می‌توانیم مقدار کران را در هر یک از رئوس درخت فضای حالت به صورت زیر محاسبه کنیم.



- مقدار کران را برای همه رئوس محاسبه می کنیم و رأسی را برای بسط دادن انتخاب می کنیم که مقدار کران آن کمینه باشد.
- در اینجا از بین رئوس $[1, 5]$, $[1, 4]$, $[1, 3]$, $[1, 2]$ رأس $[1, 3]$ کوچکترین کران را دارد که برابر با ۲۲ است.
- وقتی رأس $[1, 3]$ بسط داده شد، در نهایت کوتاهترین طول مسیری که در ریشه به دست می آید برابر با ۳۱ است.
- اما در رأس $[1, 4]$ همچنان یک کران کوچکتر برابر با ۳۰ وجود دارد، بنابراین آن رأس را بسط می دهیم و در نهایت مسیری با طول ۳۰ پیدا می کنیم.

- هر رأس در درخت فضای حالت را می توانیم با ساختمان زیر نشان دهیم.

Algorithm Node Structure

```
struct node
1:   int level  ▷ the node's level in the tree
2:   ordered-set path
3:   int bound
```

مسئله فروشنده دوره گرد

- در الگوریتم زیر برای یک گراف با n رأس، که وزن یال‌های آن با ماتریس W داده شده است، می‌خواهیم کوتاهترین دور همیلتونی را پیدا کنیم.

Algorithm Travelling Salesman Problem

```
function TRAVELLING-SALESMAN-PROBLEM(int n, int-matrix W)
1: ordered-set opttour ▷ Optimal tour
2: int minlength =  $\infty$  ▷ The length of the optimal tour
3: priority-queue PQ
4: node v
5: initialize(PQ) ▷ Initialize PQ to be empty.
6: v.level = 0
7: v.path = [1] ▷ Make first vertex the
8: v.bound = bound(v) ▷ starting one.
9: insert(PQ, v)
```

Algorithm Travelling Salesman Problem

```

10: while !empty(PQ) do
11:   remove(PQ, v)    ▷ Remove node with best bound.
12:   if (v.bound < minlength) then
13:     for (all i not in v.path) do
14:       create the new tree node u
15:       u.level = v.level + 1          ▷ Set u to a child of v.
16:       u.path = [v.path , i]          ▷ put i at the end of v.path
                                       ▷ Check if next vertex completes a tour.
17:       if (u.level == n-2) then
         ▷ put the last vertex not in u.path and also vertex 1 at the
         end of u.path
18:       u.path = [u.path, last-vertex, 1]
         ▷ Function length computes the length of the tour.
19:       if (length(u) < minlength) then
20:         minlength = length(u)
21:         opttour = u.path
22:       else
23:         u.bound = bound(u)
24:         if (u.bound < minlength) then
25:           insert(PQ, u)
26: return (opttour, minlength)

```

- کران یک رأس درخت فضای حالت را به صورت زیر محاسبه می کنیم.

Algorithm Computing Bound for Travelling Salesman Problem

```
function BOUND(node v)
1: bound = 0
2: for edge (i,j) in v.path do
3:   bound += length(i,j)
4: i = last vertex in v.path
5: min =  $\min_k \{ W[i][k], \text{vertex } k \notin v.\text{path} \}$ 
6: bound += min
7: for vertex i  $\notin v.\text{path}$  do
8:   min =  $\min_k \{ W[i][k], \text{vertex } k \neq i \text{ and } k \notin v.\text{path} \text{ or } k = 1 \}$ 
9:   bound += min
10: return bound
```
